Comunicación inalámbrica con NRF24L01 (TX/RX) – ESP32 / Arduino

Fecha: 11/03/2026

Integrantes:

• Eduardo Cadengo López

• Itzel Citlalli Martell De La Cruz

• Damian Alexander Diaz Piña

Indice

Descripción de la ESP32 1

Descripción del NRF24L01 1

Diferencias entre NRF24L01 “normal” y versiones PA+LNA / con antena externa 2

Componentes de electrónica discreta (LED, botón, resistencias) 2

Materiales que usamos 2

Tablas de conexión (Diagrama de conexion) 3

Código comentado (ESP32 -- ESP32): 4

Procedimiento de prueba (paso a paso) 7

Conclusiones personales 8

URL a GitHub: 8

URL a evidencia en video: 9

Evidencia fotos: 9

Descripción de la ESP32

La ESP32 es un SoC con MCU dual-core Tensilica LX6 (hasta ~240 MHz), WiFi 2.4 GHz y Bluetooth BLE integrados, GPIOs multifunción, ADC/DAC, I²C, SPI, UART, PWM, etc.
Para esta práctica la usamos como controlador SPI del NRF24L01 y para manejar la entrada del botón (TX) y la salida del LED (RX).
SPI típico en ESP32 (VSPI): SCK=GPIO18, MISO=GPIO19, MOSI=GPIO23, SS(=CSN)=GPIO5. CE es un GPIO libre (usaremos GPIO4).

Descripción del NRF24L01

El NRF24L01+ es un transceptor de 2.4 GHz (ISM) con interfaz SPI, payload hasta 32 bytes, 6 pipes de recepción con direcciones, auto-ACK y reintentos, 125 canales (0–124) y tasas de datos de 250 kbps, 1 Mbps y 2 Mbps. Para maximizar alcance/robustez usamos canal fijo y, si se desea, 250 kbps. Se alimenta a 3.3 V (nunca 5 V). Es muy recomendable

desacoplar con un capacitor de 10–47 µF entre 3.3 V y GND del módulo.

Diferencias entre NRF24L01 “normal” y versiones PA+LNA / con antena externa

Variante	Antena	Potencia / Sensibilidad	Consumo típico	Alcance	Cuando usar
NRF24L01	PCB integrada	Potencia baja, sensibilidad estándar	~12–15 mA TX/RX	Corto/medio (decenas de metros en interior)	Prácticas de mesa y distancias cortas
NRF24L01+ PA+LNA	SMA externa	Mayor potencia de TX y mejor sensibilidad RX	Picos >100 mA en TX	Medio/largo (cientos de metros en línea de vista)	Cuando requieres alcance/robustez; demanda 3.3 V estable y buena desacoplación

Componentes de electrónica discreta (LED, botón, resistencias)

- LED (receptor): Diodo emisor de luz que enciende al aplicar corriente directa. Se conecta con resistencia limitadora para limitar la corriente (ej. 220 Ω a 3.3 V ≈ 5–10 mA).
- Botón (transmisor): Actúa como entrada digital. Usamos INPUT_PULLUP para tener nivel alto por defecto; al presionar a GND, lee LOW (activo).
- Resistencia de 220 Ω: Limita corriente del LED.

Materiales que usamos

- 2 × Módulo NRF24L01
- 2 × ESP32 (o 1 × ESP32 + 1 × Arduino UNO)
- 1 × PushButton (TX)
- 1 × LED (RX)
- 1 × Resistencia 220 Ω
- Jumpers hembra–hembra / hembra–macho
- 1 x Protoboard

Tablas de conexión (Diagrama de conexion)

Señales del NRF24L01

- **VCC - 3.3 V** (no 5 V)
- **GND - GND**
- **CE - GPIO configurable**
- **CSN (CS/SS) - pin SS del SPI**
- **SCK - reloj SPI**
- **MOSI - SPI MOSI**
- **MISO - SPI MISO**
- **IRQ** (opcional, no usado)

Transmisor: ESP32 como TX (con botón)

NRF24L01	ESP32 (TX)
CE	**GPIO 4**
CSN	**GPIO 5**
SCK	**GPIO 18**
MOSI	**GPIO 23**
MISO	**GPIO 19**
VCC	**3.3 V**
GND	**GND**

Botón (TX):

- Un pin del botón a GPIO 14
- El otro pin del botón a GND
- Configurado como INPUT_PULLUP (no requiere resistencia externa)

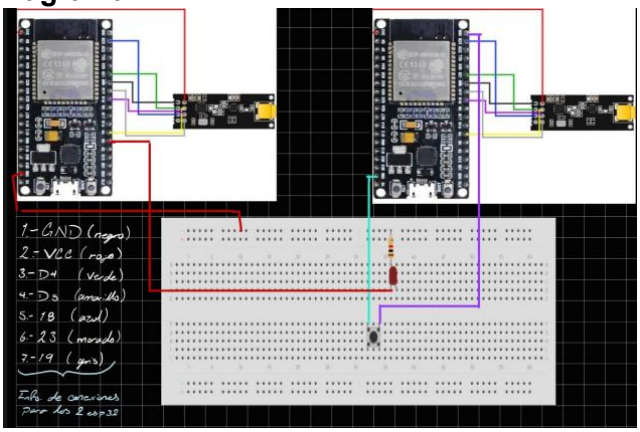
Receptor: ESP32 como RX (con LED)

NRF24L01	ESP32 (RX)
CE	**GPIO 4**
CSN	**GPIO 5**
SCK	**GPIO 18**
MOSI	**GPIO 23**
MISO	**GPIO 19**
VCC	**3.3 V**
GND	**GND**

LED (RX):

- Ánodo del LED -- GPIO 2 a través de resistencia 220 Ω
- Cátodo del LED – GND

Diagrama:



Código comentado (ESP32 -- ESP32):

```
Transmisor (ESP32 – con botón en GPIO 14)

// --- TRANSMISOR ESP32 ---
// Envia 'mensaje=1' repetidamente mientras el boton está PRESIONADO (LOW)

#include <SPI.h>
#include <RF24.h>
```

```
#define CE_PIN   4
#define CSN_PIN  5
#define BOTON   14   // Boton a GND, entrada con pull-up

// SPI VSPI: SCK=18, MISO=19, MOSI=23, SS=5
RF24 radio(CE_PIN, CSN_PIN);

// Dirección de 5 bytes (RF24 usa 5 bytes efectivos)
const byte direccion[6] = "00001";

void setup() {
  Serial.begin(115200);
  delay(200);

  pinMode(BOTON, INPUT_PULLUP);

  // Inicializa SPI en ESP32 con pines explícitos
  SPI.begin(18, 19, 23, 5);

  Serial.println("Iniciando TRANSMISOR...");
  if (!radio.begin()) {
    Serial.println("ERROR: NRF24 no detectado en TX");
    while (1);
  }
  Serial.println("NRF24 OK (TX)");

  // Configuración de radio (igual que en RX)
  radio.setChannel(13);              // mismo canal en TX/RX
  radio.setPALevel(RF24_PA_LOW);     // ajusta si hace falta alcance
  radio.setDataRate(RF24_250KBPS);   // robustez


  radio.openWritingPipe(direccion);
  radio.stopListening();             // Modo TX

  Serial.println("TX listo. Presiona el boton para transmitir...");
}

void loop() {
  int estado = digitalRead(BOTON);

  if (estado == LOW) { // activo
    int mensaje = 1;
    bool ok = radio.write(&mensaje, sizeof(mensaje));
```

```
    if (ok) {
      Serial.println("TX: paquete enviado (1)");
    } else {
      Serial.println("TX: ERROR al enviar");
    }
    delay(35); // periodo corto para que el RX mantenga el LED encendido
  } else {
    // No enviamos nada; el RX apagará el LED tras el timeout
    delay(10);
  }
}
```

Receptor (ESP32 – con LED en GPIO 2)

```
// --- RECEPTOR ESP32 ---
// Enciende LED mientras lleguen paquetes con 'mensaje=1'.
// Si no llegan paquetes por 'TIMEOUT_MS', apaga el LED.

#include <SPI.h>
#include <RF24.h>

#define CE_PIN   4
#define CSN_PIN  5
#define LED_PIN  2

RF24 radio(CE_PIN, CSN_PIN);
const byte direccion[6] = "00001";

const unsigned long TIMEOUT_MS = 200;  // ventana sin paquetes para apagar
LED
unsigned long ultimoPaqueteMs = 0;

void setup() {
  Serial.begin(115200);
  delay(200);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  SPI.begin(18, 19, 23, 5);

  Serial.println("Iniciando RECEPTOR...");
  if (!radio.begin()) {
```

```
    Serial.println("ERROR: NRF24 no detectado en RX");
    while (1);
  }
  Serial.println("NRF24 OK (RX)");

  radio.setChannel(13); // número del canal, debe ser igual al del
Transmisor
  radio.setPALevel(RF24_PA_LOW);
  radio.setDataRate(RF24_250KBPS);

  radio.openReadingPipe(0, direccion);
  radio.startListening();

  Serial.println("RX listo. Esperando paquetes...");
}

void loop() {
  // Revisión de paquetes disponibles
  while (radio.available()) {
    int mensaje = 0;
    radio.read(&mensaje, sizeof(mensaje));
    Serial.print("RX: paquete recibido -> ");
    Serial.println(mensaje);

    if (mensaje == 1) {
      digitalWrite(LED_PIN, HIGH);        // enciende LED
      ultimoPaqueteMs = millis();         // reinicia ventana de tiempo
    }
  }

  // Control de timeout: si pasa el tiempo sin paquetes, apaga LED
  if (digitalRead(LED_PIN) == HIGH) {
    if (millis() - ultimoPaqueteMs > TIMEOUT_MS) {
      digitalWrite(LED_PIN, LOW);
      Serial.println("RX: timeout sin paquetes, LED apagado");
    }
  }
}
```

Procedimiento de prueba (paso a paso)

1. **Cableado**: Conecta NRF24L01 en TX y RX según tablas de conexión.
2. **Cargar código**:
    - Sube el TX al ESP32 con botón.

- Sube el RX al ESP32 con LED.

3. **Monitor serial (RX)** a **115200 baud**, **Debes ver:**
    - "NRF24 OK (RX)" - radio inicializado.
    - "RX listo. Esperando paquetes..."
4. **Prueba del botón**:
    - Presiona el botón en TX. En RX verás "paquete recibido" y el LED permanecerá encendido mientras sostengas el botón.
    - Suelta el botón: después de ~0.2 s el LED se apaga y dejan de llegar mensajes.

Conclusiones personales

Damian: Durante esta práctica entendí que la comunicación inalámbrica no solo depende del código, sino también de detalles pequeños como el canal, la dirección y la estabilidad del voltaje del módulo. Al principio pensé que todo iba a funcionar con solo conectar y subir el programa, pero me di cuenta de que revisar cada conexión y mantener la misma configuración en TX y RX es clave. También comprobé que el LED no se mantiene encendido por arte de magia; se necesita programar la lógica correcta para interpretar los mensajes y controlar el tiempo sin recepción. En general, la práctica me ayudó a comprender de forma más realista cómo funciona un enlace simple entre dispositivos.

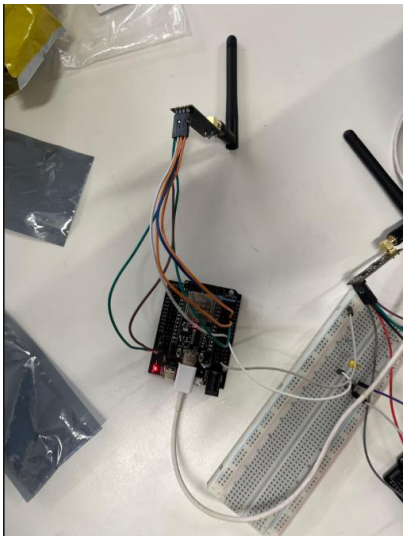
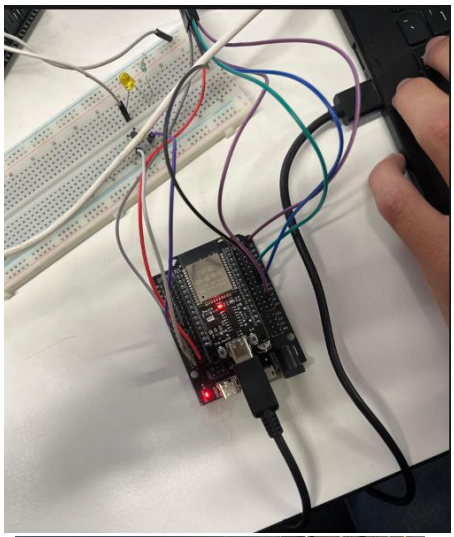
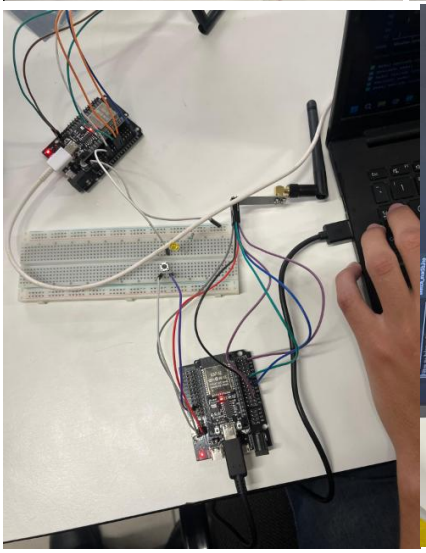
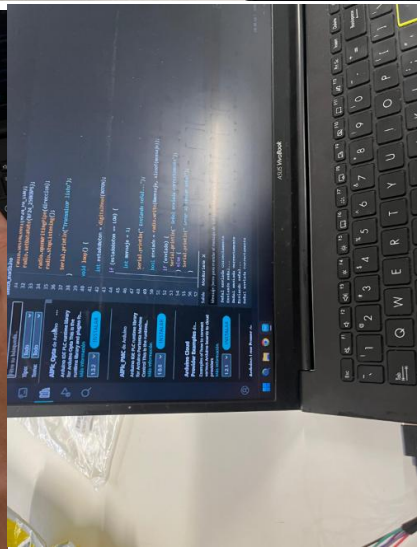
Eduardo: Esta actividad me ayudó a perderle el miedo a trabajar con módulos externos y a entender mejor cómo se comunican los microcontroladores entre sí. Antes veía al NRF24L01 como algo complicado, pero ya usando el SPI, los pipes y el canal, entendí que todo es cuestión de seguir un orden. Me gustó especialmente ver cómo el botón en un dispositivo podía activar un LED en otro sin cables de por medio. Fue una buena forma de conocer la lógica de transmisión y recepción, y de ver en acción conceptos que normalmente se quedan en teoría.

Itzel: Lo que más me dejó esta práctica fue la importancia de experimentar y no rendirse cuando algo no funciona a la primera. Hubo momentos en los que pensé que el módulo estaba dañado o que el código tenía errores graves, pero al final era simplemente un cable mal puesto o un canal mal configurado. También aprendí que los módulos inalámbricos son sensibles a la alimentación y requieren una buena estabilidad para trabajar bien. Al final, ver cómo el receptor encendía el LED justo cuando el transmisor mandaba los mensajes fue una sensación de logro que hizo que valiera la pena todo el proceso.

URL a GitHub:

https://github.com/AresGodKiller/Tecnologias_Inalambricas/tree/main/2_Practica%20NFR24L01_EnvioYLed

URL a evidencia en video:

https://youtube.com/shorts/ukE-nj5UNh0?feature=share

Evidencia fotos: